

1. Program Overview

My program is a 9x9 sudoku generator that generates a fully solved Sudoku puzzle upon running it.

2. Data Structures

2D Array:

- I used a 2D array to store the completed Sudoku puzzle. The 2D array in my code is named "grid". I filled the grid in the boxGen method after I had already generated the random order for the nine values of each box, and I filled each cell of each row in the grid individually in the rowGen method (grid[i][j]).

ArrayList:

- I used many ArrayLists in my code, as I found them effective in generating each set of nine numbers to create the first three boxes, the options for each cell when generating the rows, and information about each row, column, and box.
- In the nineGen method, I used an ArrayList: ArrayList<Integer> current = new ArrayList<Integer>();. This allowed me to use the ArrayList indexOf method to check if the random number from 1 to 9 I was generating was already a part of the randomly generated nine numbers, guaranteeing a unique ordering of nine numbers every time.
- I used three ArrayLists in the rowGen method to store the current numbers in the corresponding rows, columns, and boxes for each cell so that I could easily verify if a new number from 1 to 9 was available to be used in the cell or not using the ArrayList method indexOf.
- I also used an ArrayList to store my options from one to nine from each cell in the rowGen method. I created a new ArrayList: ArrayList<Integer> options = new ArrayList<Integer>();, and went through each number from 1 to 9. If it was not present in the row, column, or box arrays for that specific cell, I added it to the options array.

3. Algorithm Design

When developing the program to generate my Sudoku puzzle, I wanted it to be efficient yet simple for me to code with the experience I have. Upon doing some research, I realized that the most efficient way to begin was to fill the three boxes that didn't depend on any others, so three in a diagonal. I used the nineGen methods and boxGen methods to accomplish this.

From there, I decided to continue by generating each individual row with a method called rowGen. For each row, my program fills it in cell by cell. If there is no valid option for a cell, it clears the row and restarts the generation.

To avoid the program running forever if there is no valid row, I implemented a regenCount variable, which essentially counts the number of times that the rowGen method has retried, and

if it is above a certain number (I chose 200), then the whole puzzle will clear and the process will restart with generating new diagonal boxes and refilling the rows. This guarantees that even if the first attempt did not work, one run of the program will produce a valid puzzle.

4. Methods

nineGen - nineGen takes the numbers from 1-9 and places them in an ArrayList in a random order using Math.random to pick a number from 1-9 and the ArrayList method indexOf in order to verify if the number is already a part of the array.

boxGen - boxGen accepts a parameter start that denotes what cell is the top left corner of the box that it is generating. It then takes an array generated from nineGen and fills it into the box that was given by the parameter.

rowGen - rowGen takes a given row as a parameter and fills in each cell of that row with a valid number from 1-9 by looping through each column for the given row.

- For each given cell, it generates an ArrayList for the numbers already in that row, column, and box, as well as an "options" ArrayList. The code loops through all of the numbers from 1-9 and adds the ones that are not present in any of the row, column, and box arrays to the options array.
- If there are options, it shuffles the options and picks a random number from it
- If there are no options, the row is completely cleared and it restarts filling the row the same way
- The method keeps track of the number of resets it has to do so that in case the number is too high (no possible row regenerations), the whole puzzle can reset

Main method - the main method generates the Sudoku puzzle, using the regenBoolean to regenerate it as needed. boxGen is called for each of the boxes, and a for loop is used for each row to fill it in. It then uses a for loop to print out the puzzle in an aesthetically pleasing format

5. Testing plan

1. I tested the nineGen method originally by using a for loop to print out the ArrayList generated by the method and ran it many times to ensure it would be a randomly ordered set of numbers 1-9.
2. Valid Sudoku grid generation - I ran the program at least 20 times to verify that it would generate a valid grid every time in a reasonable amount of time.
3. regenCount checker - When I was deciding on a threshold for the regenCount, I ran the code many times and printed the regenCount for each attempt so I could get an understanding of the average number of regenerations for successful puzzles. I later deleted this line

6. Challenges and Solutions

One challenge that I faced was the fact that although my row would regenerate if there were no valid options for a cell, there was originally no way to undo previous rows if there were no possible row permutations. The only way that I saw to backtrack was to completely change my row regeneration logic. Instead of doing that, I decided to completely erase the puzzle if the code detected that it was taking too many row regeneration attempts. I implemented a `regenCount` variable and `regenBoolean` boolean that allowed me to count the number of regenerations, and if it went above a certain threshold based on my testing, restarted the entire regeneration process. Although this may not be the most optimal way to generate a Sudoku puzzle, I found that it generated quickly and successfully every time and worked with the logic that I had written the entire program with, which was a good solution for me.